

MIAE 215 · Getting Started with C++ Programming

Master Study Guide & Comprehensive Mini-Course Reference

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Instructor: Prof. Brandon W. Gordon · Course: MIAE 215 / MECH 215 · Language: ISO Standard C++

Table of Contents

1. [Academic Orientation & The 2-Week Learning Roadmap](#)
2. [Software Toolchain & IDE Setup Guide](#)
3. [Lesson 2: Anatomy & Execution of a C++ Program](#)
4. [Lesson 3: Fundamental Variable Types, Memory Storage & Limits](#)
5. [Lesson 4: Expressions, Operators & Precedence Rules](#)
6. [Lesson 5: Control Statements, Decision Logic & Loops](#)
7. [Lesson 6: Fixed-Size Arrays & Numerical Data Buffers](#)
8. [Master Solved Exercises Catalog: Lessons 3 Through 6](#)
9. [Official Troubleshooting Playbook & Environment Recovery](#)
10. [Exam Competency Matrix & Golden Coding Rules](#)

1. Academic Orientation & The 2-Week Learning Roadmap

The **MIAE 215 Online Mini-Course** is a mandatory educational bridge created by Prof. Brandon W. Gordon to ensure every engineering student establishes robust coding fundamentals before formal lectures and laboratory sessions commence.

In the Concordia Department of Mechanical, Industrial & Aerospace Engineering, programming is not approached as an abstract branch of pure computer science. It is the operational language of modern engineering: **physical automation, digital sensors, numerical finite element simulations, and robotic control.**

MIAE 215 TWO-WEEK LEARNING SCHEDULE		
Timeline	Focus & Milestone Deliverables	Expected Hours
Week 1	• Lesson 1: Software Environment & MinGW	1.5 – 2.0 hrs
	• Lesson 2: First Programs, main(), Streams	1.5 – 2.0 hrs
	• Lesson 3: Primitive Types & Memory Sizes	1.5 – 2.0 hrs
	• Lesson 4: Expressions, Operators, Preced.	1.5 – 2.0 hrs
Week 2	• Lesson 5: Decision Logic, if/else, Loops	1.5 – 2.0 hrs
	• Lesson 6: Linear Arrays & Buffer Math	1.5 – 2.0 hrs
Total Time Commitment: 8 to 10 Hours of Deep Practice		

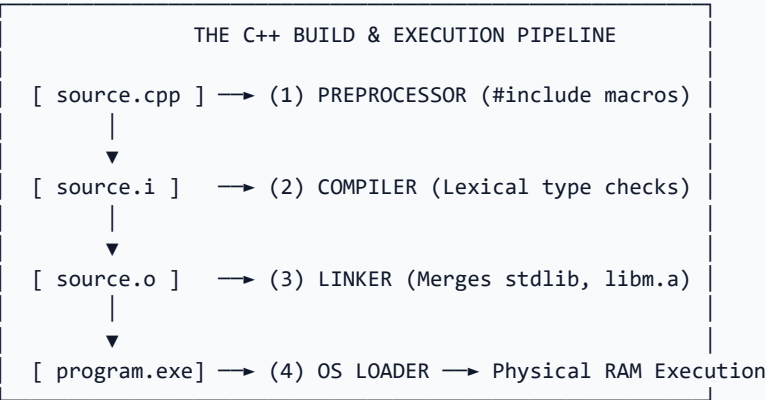
Why Engineers Must Master C++

- **Embedded Hardware & Microcontrollers:** High-level scripting languages such as Python or MATLAB are unsuitable for deterministic, hard-real-time motor control. C++ compiles directly into native CPU machine opcodes, providing sub-microsecond responsiveness.
- **Academic Project Integration:** In departmental capstone courses such as **MECH 390** (autonomous mobile robotics) and **MECH 490** (multidisciplinary mechatronics capstone), students construct micro-controlled robotic systems powered by C++ running on 8-bit AVR Arduinos and 32-bit ARM processors.
- **Student Engineering Societies:** Key competitive organizations including **Space Concordia** (rocketry, rovers, CubeSats) and **SAE Concordia** (Formula Racing, Aero Design) write performance-critical telemetry acquisition and flight software in modern C++.

Key Takeaway for Beginners:
A large number of students struggle in MIAE 215 simply due to a lack of early programming familiarity. The small investment of 8 to 10 hours spent working through these six lessons eliminates this hurdle and builds permanent academic confidence.

2. Software Toolchain & IDE Setup Guide

To write and execute C++ programs on a local machine, engineers utilize an **Integrated Development Environment (IDE)** bundled with a native C++ compiler.



IDE Platform Comparison Matrix

IDE / Environment	Supported Operating Systems	Primary Strengths	Recommendation for MIAE 215
Code::Blocks 17.12 / 20.03	Windows, Linux	Lightweight, fast installation, transparent compiler configuration, bundled with MinGW.	Top Recommendation for all Windows users.
CodeLite	Windows, macOS, Linux	Clean interface, excellent cross-platform support on Apple silicon via LLVM/Clang.	Top Recommendation for macOS users.
Visual Studio Community	Windows	Industry-standard debugger, advanced profiling tools, enterprise-grade autocomplete.	Recommended for students planning advanced software electives.
CLion (JetBrains)	Windows, macOS, Linux	Deep semantic code inspection, CMake integration (free educational license).	Excellent for experienced programmers.

Standard Code::Blocks Installation on Windows

1. Download the installer bundle with the exact name containing `mingw`: `codeblocks-17.12mingw-setup.exe` or `codeblocks-20.03mingw-setup.exe`.
2. Do **not** select the smaller installer without MinGW; that file provides only the text editor and will fail to compile programs.
3. Accept the default installation directory: `C:\Program Files\CodeBlocks`.
4. If Code::Blocks reports "Can't find compiler executable in your configured search path":
 - Navigate to **Settings** → **Compiler...** → **Selected Compiler: GNU GCC Compiler**.
 - Switch to the **Toolchain Executables** tab.
 - Verify that the path points to `C:\Program Files\CodeBlocks\MinGW` and click **Auto-detect**.

Standalone Portable Alternative

For students operating on university lab computers or machines with restricted administrator privileges, a pre-configured portable distribution is provided inside this repository at: [05 - Software & Flowcharts/CodeBlocks_17.12_portable/codeblocks.exe](#)

This package requires zero system installation and runs directly from any local folder or USB flash drive.

3. Lesson 2: Anatomy & Execution of a C++ Program

In Lesson 2, Prof. Gordon introduces the fundamental structure required by every compliant C++ executable.

```
#include <iostream> // Directs preprocessor to load standard stream I/O definitions
#include <cstdio>    // Directs preprocessor to load standard C I/O (getchar)

using namespace std; // Exposes standard library identifiers into the global namespace

int main()          // Mandatory program entry point function
{
    // Output message to console stream
    cout << "Hello World!\n";

    // Pause terminal execution to prevent instant window closure
    cout << "\nPress enter to continue.";
    getchar();

    return 0;        // Return code 0 communicates clean execution to the operating system
}
```

Component-by-Component Dissection

1. `#include <iostream>`: The `#` character flags a preprocessor directive. The preprocessor physically inserts the declarations of stream objects (`cout`, `cin`) into the translation unit before the compiler evaluates syntax.
2. `#include <cstdio>`: Contains declarations for lower-level C runtime routines, specifically `getchar()`. Omitting `<cstdio>` triggers compiler errors on strict modern ISO C++ compilers.
3. `using namespace std;`: Prevents repetitive namespace prefixing. Without this declaration, every standard stream requires explicit qualification: `std::cout`, `std::cin`, `std::endl`.
4. `int main()`: The universal entry point of every C++ executable. The CPU transfers execution to the first instruction inside `main()`. Returning `int` allows the process to report termination status back to the host operating system.
5. **The Stream Insertion Operator (<<)**: Directs character data sequentially into the `cout` buffer.
6. `\n` vs. `endl`: `\n` inserts a line feed into the stream buffer. `endl` inserts a line feed **and** forces an immediate hardware buffer flush (`fflush`), which can significantly degrade performance inside high-frequency numerical loops.

The Console Flash Trap

When executing a compiled binary (`program.exe`) directly by double-clicking in Windows Explorer:

- Windows spawns a temporary console terminal.
- The CPU runs all instructions in fractions of a millisecond.
- Upon reaching `return 0;` , the operating system closes the window immediately before the human eye can register the output.
- **The Solution:** Always invoke `getchar();` immediately prior to `return 0;` . This suspends program execution until the user registers an Enter keystroke.

4. Lesson 3: Fundamental Variable Types, Memory Storage & Limits

A variable is a **named, typed memory storage cell** allocated at a physical byte address in RAM. C++ is statically typed: every variable must have its exact data type and memory footprint established at compile time.

Master Primitive Types Matrix

Data Type	Keyword	Hardware Size	Binary Storage Scheme	Numerical Value Range	Typical Engineering Usage
Character	<code>char</code>	1 byte (8 bits)	Two's Complement Integer	−128 to +127 (ASCII code)	Keyboard keys, ASCII text, single raw telemetry bytes.
Integer	<code>int</code>	4 bytes (32 bits)	Two's Complement Integer	−2,147,483,648 to +2,147,483,647	Loop counters, array indices, discrete states.
Single-Precision	<code>float</code>	4 bytes (32 bits)	IEEE 754 Floating Point	$\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{+38}$ (~7 digits)	Embedded microcontrollers with constrained RAM.
Double-Precision	<code>double</code>	8 bytes (64 bits)	IEEE 754 Floating Point	$\pm 2.23 \times 10^{-308}$ to $\pm 1.79 \times 10^{+308}$ (~16 digits)	Standard choice for engineering simulations and physics.
Boolean	<code>bool</code>	1 byte (8 bits)	Single-bit Flag in Byte	<code>true</code> (1) or <code>false</code> (0)	Decision logic, conditional flags, sensor triggers.

Memory Byte Layout in RAM (Addresses increment from left to right):

char (1 byte) : [Byte 0]

int (4 bytes): [Byte 0][Byte 1][Byte 2][Byte 3]

double (8 bytes): [B0][B1][B2][B3][B4][B5][B6][B7]

Inspecting Memory Footprints with `sizeof()`

```
#include <iostream>
using namespace std;

int main() {
    int count = 100;
    cout << "sizeof(count) = " << sizeof(count) << " bytes\n"; // 4 bytes
    cout << "sizeof(double) = " << sizeof(double) << " bytes\n"; // 8 bytes
    return 0;
}
```

The Uninitialized Variable Trap

In C++, declaring a local variable does **not** initialize its memory to zero:

```
int q;
cout << "Value of q: " << q; // DANGER: Prints undefined stack garbage!
```

When memory is allocated on the runtime Stack, `q` assumes whatever leftover bit pattern occupied those RAM bytes from prior system operations. Always initialize explicitly: `int q = 0;` or `double x = 0.0;`.

Floating-Point Limits: Overflow and Underflow

- **Overflow:** Multiplying positive numbers beyond the hardware representable double ceiling ($\approx 1.8 \times 10^{308}$) sets the variable to `inf` (infinity):

```
double x = 1.0e308;
x = x * 10.0; // Overflows -> inf
```

- **Underflow:** Dividing values smaller than the minimum representable positive normalized magnitude ($\approx 2.2 \times 10^{-308}$) collapses silently to `0.0`:

```
double z = 1.0e-306;
z = z / 1.0e10; // Underflows -> 0.0
```

Keyboard Stream Input via `cin >>`

```
int id;
double voltage;
cout << "Enter sensor ID and measured voltage: ";
cin >> id >> voltage; // Reads tokens separated by spaces, tabs, or newlines
```

5. Lesson 4: Expressions, Operators & Precedence Rules

An **operator** performs a specific computation on one or more variables (**operands**). An **expression** is a syntactic combination of operators and operands that evaluates to a concrete value.

C++ OPERATOR PRECEDENCE

1. () Parentheses (Highest Priority)
2. Postfix Operators: `x++`, `x--`
3. Unary Operators: `++x`, `--x`, `-x`, `(cast)`
4. Multiplicative Arithmetic: `*`, `/`, `%`
5. Additive Arithmetic: `+`, `-`
6. Relational Comparisons: `<`, `<=`, `>`, `>=`
7. Equality Comparisons: `==`, `!=`
8. Logical AND: `&&`
9. Logical OR: `||`
10. Assignment Operators: `=`, `+=`, `-=`, `*=`, `/=` (Lowest)

The Imperative Assignment Operator (`=`)

In pure algebra, $x = y$ defines a symmetric identity ($y = x$). In C++, `=` is an **imperative memory copy instruction**:

```
x = 3;
x = 2 * x + 1; // Evaluates RHS: 2*(3) + 1 = 7, then overwrites x with 7
```

This is **sequential memory mutation**, evaluated line-by-line.

Chained Assignments

The assignment operator has **right-to-left associativity**:

```
int a, b, c;
a = b = c = 10; // Evaluates: c = 10, then b = 10, then a = 10
```

The #1 Trap: Truncated Integer Division

When both operands of the division operator `/` are integers, C++ performs truncated integer division, discarding all fractional decimals:

```
int a = 1, b = 3;
double x = a / b; // BUG: 1 / 3 evaluates to integer 0, which widens to 0.0!

// The Engineering Remedy: Ensure at least one operand is floating-point:
double y = 1.0 / 3.0; // Evaluates to 0.3333333333333333
double z = (double)a / b; // Explicit cast promotes operation to double
```

The Modulus Operator (`%`) & Discrete Algorithms

The modulus operator (`%`) calculates the integer remainder after division:

$$a = b \cdot q + r \quad (0 \leq r < |b|)$$

Note: Modulus is defined **strictly for integer data types** in C++.

Pattern A: Reversing & Peeling Decimal Digits Right-to-Left

```
int b = 237;

int r1 = b % 10; // 237 % 10 = 7 (extracts units digit)
b = b / 10;      // 237 / 10 = 23 (truncates units digit)

int r2 = b % 10; // 23 % 10 = 3 (extracts tens digit)
b = b / 10;      // 23 / 10 = 2

int r3 = b % 10; // 2 % 10 = 2 (extracts hundreds digit)
```

Pattern B: Cyclic Ring Buffer Wrapping

```
// Wraps index smoothly within [0, BUFFER_SIZE - 1]:
index = (index + 1) % BUFFER_SIZE;
```

Pattern C: Alternating Numerical Sequences $(-1)^n$

```
// Yields +1 for even n, -1 for odd n:
int sign = (n % 2 == 0) ? 1 : -1;
```

Floating-Point Base-2 Roundoff & Safe Epsilon Comparison

Computers store numbers in base-2 binary. Fractions such as 0.1 or transcendental values like π cannot be represented with finite binary digits:

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double pi = 4.0 * atan(1.0);
    double val = cos(pi / 2.0); // Mathematically 0.0
    cout << "cos(pi/2) = " << val << "\n"; // Prints: 6.12323e-17 (NOT 0.0!)
    return 0;
}
```

Teacher Exam Warning / Pitfall:

Never test floating-point numbers for exact equality using `==` ! Always compare against an **engineering tolerance threshold** (ϵ):

```
double eps = 1.0e-9;
if (abs(val - 0.0) < eps) {
    cout << "Value is physically zero within tolerance.\n";
}
```

6. Lesson 5: Control Statements, Decision Logic & Loops

Control statements govern the dynamic execution pathway of a program based on runtime Boolean conditions.

Selection Structures: `if`, `if-else`, and Cascading `else if`

```
double distance_to_wall = 0.42;

if (distance_to_wall > 1.0) {
    cout << "Path clear. Full speed ahead.\n";
} else if (distance_to_wall > 0.5) {
    cout << "Approaching obstacle. Decelerating motors.\n";
} else {
    cout << "Obstacle imminent! Halting and engaging emergency brake.\n";
}
```

The Catastrophic Assignment in Condition Bug

- `if (x == 5)` : Evaluates whether `x` is equal to 5.
- `if (x = 5)` : **Overwrites `x` with 5**, and since 5 is non-zero, the condition evaluates to `true` every single time!

Relational & Logical Operators

Operator	Type	Meaning	Truth Requirement
> , <	Relational	Greater than, Less than	Mathematical comparison.
>= , <=	Relational	Greater or equal, Less or equal	Mathematical comparison.
== , !=	Relational	Equality, Inequality	Exact bit comparison.
&&	Logical	Logical AND	True only if both operands are true.
	Logical	Logical OR	True if at least one operand is true.
!	Logical	Logical NOT	Inverts Boolean truth value (!true == false).

Short-Circuit Evaluation & Zero-Division Safety

Logical expressions evaluate strictly from left to right and terminate immediately once the final outcome is guaranteed:

```
double denominator = 0.0;
double numerator = 100.0;

// SHORT-CIRCUIT SAFETY GUARD:
if (denominator != 0.0 && (numerator / denominator > 5.0)) {
    cout << "Ratio exceeded threshold.\n";
}
```

Because `denominator != 0.0` evaluates to `false`, the right-hand division `numerator / denominator` is **never executed**, completely preventing a fatal hardware division-by-zero crash.

Iterative Loops: `for` and `while`

The `for` Loop Architecture

```
for (initialization; test_condition; step_update) {
    // Repeated loop body
}
```

Crucial Exam Concept: Post-Loop Iterator State Analysis

```
int i;
for (i = 10; i > -1; i--) {
    // Loop body executes for i = 10, 9, 8 ... 0
}
cout << "Post-loop i = " << i << "\n"; // PRINTS: -1 !
```

The loop body executes while `i > -1` remains `true` (down to `i = 0`). When `i` decrements from `0` to `-1`, the test `-1 > -1` evaluates to `false`, halting the loop. Therefore, the post-loop value of `i` stored in memory is `-1`.

7. Lesson 6: Fixed-Size Arrays & Numerical Data Buffers

An array is a contiguous, sequential block of memory allocated to store multiple elements of the identical data type under a single identifier.

Array 'double sensor_readings[4]' in RAM ($4 * 8 = 32$ contiguous bytes):

Index 0	Index 1	Index 2	Index 3
[12.4 V]	[14.1 V]	[11.8 V]	[13.5 V]

Address: 0x100 Address: 0x108 Address: 0x110 Address: 0x118

Zero-Based Indexing & Memory Offset Formula

In C++, array indices run strictly from 0 to $N - 1$:

$$\text{Address of } A[i] = \text{Base Address} + i \times \text{sizeof}(\text{Type})$$

```
double A[3]; // Allocates 3 doubles: A[0], A[1], A[2]
A[0] = 1.25;
A[1] = 2.25;
A[2] = 3.25;
```

The Out-of-Bounds Memory Corruption Trap

Standard C++ prioritizes raw hardware execution speed over runtime overhead. Consequently, **the compiler performs no automatic array bounds checking**:

```
double A[3]; // Valid elements: A[0], A[1], A[2]
A[3] = 99.9; // CATASTROPHIC MEMORY VIOLATION!
```

Accessing `A[3]` writes 8 bytes into whatever memory immediately follows the array. This corrupts adjacent variables, invalidates CPU stack return pointers, or triggers an operating system segmentation fault (`SIGSEGV`).

Common Array Accumulation Algorithm: Sum & Average

```
double readings[5] = {10.5, 12.2, 11.0, 13.8, 12.5};
double sum = 0.0;

for (int i = 0; i < 5; i++) {
    sum += readings[i];
}

double average = sum / 5.0;
cout << "Mean Sensor Reading: " << average << " V\n";
```

8. Master Solved Exercises Catalog: Lessons 3 Through 6

Below is a detailed analysis of the official exercise problems provided across the mini-course curriculum:

Exercise Set 4: Expressions & Operators (Lesson 4)

```
int q;
double x, y, z = 0;
double d, r1 = 1.234567890123456, r2 = 1.234567890123455;

q = 7 - (7 / 3) * 3; // Integer division (7/3) = 2 -> 2*3 = 6 -> 7 - 6 = 1. Result: q = 1
x = 1 / 3 * 10.0;    // Integer division (1/3) = 0 -> 0 * 10.0 = 0.0. Result: x = 0.0
y = -1.0 / z;        // Division by double zero -> Result: y = -inf
d = r1 - r2;         // IEEE 754 precision difference -> Result: d = 1.0e-15
```

Exercise Set 5: Control Statements (Lesson 5)

```
int i;
double x = 1.1, y = 0.25, z = -3.0;

// Evaluation: (x/y) = (1.1 / 0.25) = 4.4 > 4 (TRUE)
if ((x / y) > 4) {
    z = abs(z); // z becomes 3.0
    z = z * z;   // z becomes 9.0
    x = -x;     // x becomes -1.1
}

// Next evaluation: (x/y) = (-1.1 / 0.25) = -4.4 > 4 (FALSE -> block skipped)
if ((x / y) > 4) x = -x;

cout << x << "\t" << y << "\t" << z << "\n";
// Output: -1.1    0.25    9

for (i = 10; i > -1; i--) { /* decrements to -1 */ }
cout << "i = " << i << "\n"; // Output: i = -1
```

Exercise Set 6: Arrays (Lesson 6)

```
int k, n = 0, m, H[100];

for (k = 1; k < 10; k++) {
    H[2 * k + 1] = 2 * k + 1;
    n++;
    m--;
}
```

- **Array Index Mapping:**
 - $k = 1 \implies 2(1) + 1 = 3 \implies H[3] = 3$
 - $k = 2 \implies 2(2) + 1 = 5 \implies H[5] = 5$
 - $k = 3 \implies 2(3) + 1 = 7 \implies H[7] = 7$
 - $k = 4 \implies 2(4) + 1 = 9 \implies H[9] = 9$
- **The $H[6]$ Trap:** What is the value of $H[6]$? Because $2k + 1$ generates exclusively **odd** indices, index 6 was **never written to!** Printing $H[6]$ outputs arbitrary, uninitialized memory garbage.
- **Variable State:** n increments 9 times ($k = 1 \dots 9$), resulting in $n = 9$. Variable m was never initialized; decrementing m simply mutates uninitialized garbage.

9. Official Troubleshooting Playbook & Environment Recovery

Prof. Gordon highlights the five most common setup failures encountered by students during the mini-course:

1. Browser Security Warning: "File can't be downloaded securely"

- **Symptom:** Modern web browsers flag `.rar`, `.exe`, or `.cpp` downloads from the university server.
- **Root Cause:** The engineering departmental server serves files via HTTP rather than HTTPS.
- **Remedy:** Right-click the warning item in your browser downloads tab and choose **Keep** (or click the three dots $\dots \rightarrow$ **Keep anyway**). The files are completely safe.

2. Video Audio Playback Failure

- **Symptom:** Lecture screencasts play with video but have silent or distorted audio.
- **Root Cause:** Lesson 1 video files contain no audio narration by design. Audio narration begins in Lesson 2. Furthermore, default Windows media players lack certain MPEG-4 audio codecs.
- **Remedy:** Download and install **VLC Media Player** (videolan.org). Always save videos to disk before playing rather than streaming inside a browser tab.

3. Missing `getchar()` or Linker Namespace Errors

- **Symptom:** Compiler reports `error: 'getchar' was not declared in this scope`.
- **Root Cause:** Newer C++ standards require standard C library functions to be explicitly exposed.
- **Remedy:** Always ensure `#include <cstdio>` is placed at the top of your program:

```
#include <cstdio>
```

4. CodeLite Terminal Not Launching on macOS

- **Symptom:** CodeLite compiles successfully on a Mac, but clicking Run produces no output window.
- **Root Cause:** macOS Gatekeeper security entitlements restrict third-party binaries from auto-launching terminals.
- **Remedy:** Install Apple Developer Command Line Tools via Terminal:

```
xcode-select --install
```

In CodeLite → **Settings** → **Build Settings**, click the green **"+" (Plus) button** to re-detect the LLVM Clang toolchain. Alternatively, execute the compiled Unix executable directly inside the project output folder.

5. Parallels Desktop VM: UNC Path Failure

- **Symptom:** GCC throws `UNC paths are not supported. Defaulting to Windows directory`.
- **Root Cause:** macOS shared folders map into Windows virtual machines via network UNC paths (`\\mac\Home\Desktop`). Compilers cannot write build artifacts across UNC shares.
- **Remedy:** Create your project workspace strictly on the virtual Windows `C:\` drive (e.g., `C:\miae215_projects\`). Never build across network-shared Mac folders.

10. Exam Competency Matrix & Golden Coding Rules

Core Topic Area	Primary Syntax / Keyword	Critical Exam Pitfall	Golden Engineering Rule
Integer Math	<code>/</code> , <code>%</code>	Truncated division: <code>1 / 3</code> evaluates to <code>0</code> .	Force floating-point literals: <code>1.0 / 3.0</code> .
Floating Comparisons	<code>==</code> , <code>!=</code>	Direct equality check fails due to base-2 roundoff.	Always compare with epsilon tolerance: <code>abs(x - target) < 1.0e-9</code> .
Loop Iterators	<code>for</code> , <code>while</code>	Forgetting that the post-loop iterator decrements past limit.	Trace termination condition: <code>for(i=10; i>-1; i--)</code> ends at <code>i = -1</code> .
Variable State	Declarations	Uninitialized variables hold unpredictable stack garbage.	Always initialize variables at declaration (<code>int x = 0;</code>).
Buffer Access	<code>arr[N]</code>	Accessing index <code>N</code> on array of size <code>N</code> corrupts RAM.	Valid array indices strictly span from 0 to $N - 1$.
Input Streams	<code>cin >></code>	Leftover newline <code>\n</code> causes subsequent <code>getchar()</code> to skip.	Call <code>getchar()</code> twice to clear buffer before pausing.

Official Conclusion & Academic Next Steps in MIAE 215

Having completed the **MIAE 215 Online Mini-Course**, you have successfully established a rigorous working mastery of core C++ syntax, memory models, conditional decision branching, and contiguous data arrays.

RECOMMENDED SEQUENTIAL STUDY PATHWAY

[Mini-Course Master Guide] (Completed Foundation)

▼

[02 - Comprehensive Topic Guides (Parts 1 to 4)]

• Part 1: Deep Memory Architecture, Linkers & Two's Complement

• Part 2: Explicit Casts, ASCII Math & Arduino Microcontrollers

• Part 3: Operator Precedence, atan2 Robotics & Peak Optimization

• Part 4: Boolean Short-Circuit Guards & Flowgorithm Visual Logic

▼

[04 - Practice Problems & Code Solutions]

• Assignment 1: Questions 1 through 7 Fully Solved & Traced

• Exercise Solutions Set 1: Master Code Analysis & Teacher Notes

▼

[03 - 1-Page Rapid Review Sheets (Parts 1 to 3)]

• High-yield exam cheatsheets (strictly 1 page each) for revision

1. **Advance to the Comprehensive Topic Guides:** Study the expanded, pedagogical topic guides in `02 - Comprehensive Topic Guides (Expanded & Intuitive)` to master the advanced concepts taught by Prof. Gordon directly inside the lecture code files.

2. **Review High-Yield Cheatsheets:** Keep the single-page summaries in `03 - 1-Page Rapid Review Sheets` on hand during practice sessions for immediate formula and limit recall.

3. **Practice Assignment & Exam Problems:** Open the Code::Blocks project files in `04 - Practice Problems & Code Solutions` to compile, trace, and debug the official solutions.

file:///C:/Users/Sharafath/Documents/iCloudDrive/Concodia/Semester%201/Miae%20215/_Source%20%26%20Archive/temp_html/MIAE%20215%2...

12/12